DETAILS

NAME:M.THANVI SRI

SECTION:CSE 1

ROLL NO:26000036

CASE STUDIES 1 – 10

CASE 1: University Scholarship Allocation

1.*Problem statement:*

Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules

2.*Objectives:*

Evaluate students based on marks, attendance, and family income.
Identify eligible students and rank them fairly.
Resolve ties using predefined rules.

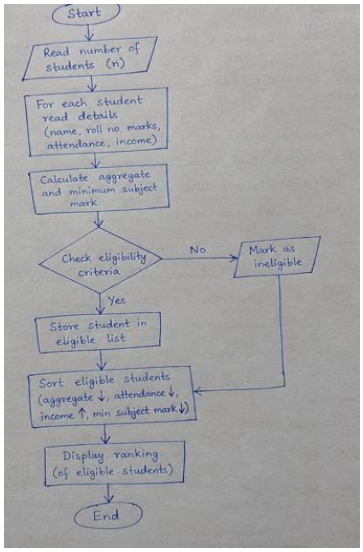
3.*Assumptions:*

1.Students must score at least 75% aggregate marks. 2. Each subject must have at least 60 marks. 3.Attendance must be at least 75%. 4.Family income must be ₹2,50,000 or less per year. 5.Students with higher marks get higher ranks.

4.*Algorithm:*

1. Start, 2. Enter student details, 3. Check marks, attendance, and family income. 4.Select eligible students, 5. Rank students according to their marks, 6.Display the results, 7.Stop.

5.FLOWCHART



6.*Psuedo code:*

```
START
INPUT student details

IF marks >= 75 AND
   subject marks >= 60 AND
   attendance >= 75 AND
   income <= 2 50000:
       PRINT "Eligible"
ELSE:
       PRINT "Not Eligible"
Rank eligible students by marks
Display results
STOP
```

7.*Python source code:*

```
n= int(input("Enter number of students: "))
students = []

for i in range(n):
 print("\nEnter student details")
 name = input("Name: ")
 marks = float(input("Aggregate marks: "))
 subject = int(input("Minimum subject marks: "))
 attendance = float(input("Attendance: "))
 income = float(input("Family income: "))

 if marks >= 75 and subject >= 60 and attendance >= 75 and
income <= 250000:
      students.append([name, marks, income])
      print("Eligible")
else:
      print("Not Eligible")

students.sort(key=lambda x: (-x[1], x[2]))

print("\nScholarship Ranking")

for rank, student in enumerate(students, start=1):
    print(rank, student[0], student[1], "%")
```

8.*Test case:*

The program is tested using five students with different marks, subject scores, attendance, and family incomes. Students who satisfy all the conditions are declared

eligible, while those who fail any condition are declared ineligible. Eligible students are ranked by higher aggregate marks, lower income, higher attendance, and lower roll.no

9.*Code and Output Screenshot:*

```
n= int(input("Enter number of students: "))
students = []

for i in range(n):
 print("\nEnter student details")
 name = input("Name: ")
 marks = float(input("Aggregate marks: "))
 subject = int(input("Minimum subject marks: "))
 attendance = float(input("Attendance: "))
 income = float(input("Family income: "))

 if marks >= 75 and subject >= 60 and attendance >= 75 and income <= 250000:
      students.append([name, marks, income])
      print("Eligible")
else:
      print("Not Eligible")

students.sort(key=lambda x: (-x[1], x[2]))

print("\nScholarship Ranking")
for rank, student in enumerate(students, start=1):
    print(rank, student[0], student[1], "%")
```

OUTPUT:

```
Enter number of students: 3

Enter student details
Name: pavani
Aggregate marks: 80
Minimum subject marks: 60
Attendance: 75
Family income: 200000
Eligible

Enter student details
Name: Leela
Aggregate marks: 85
Minimum subject marks: 60
Attendance: 60
Family income: 210000

Enter student details
Name: Bhavigna
Aggregate marks: 75
Minimum subject marks: 60
Attendance: 50
Family income: 300000
Not Eligible

Scholarship Ranking
1 pavani 80.0 %
```

10.*Conclusion:*

The program checks scholarship eligibility using marks, attendance, and income. It ranks eligible students and makes scholarship selection easier and fairer.

CASE 2: Progressive Electricity Billing

1.*Problem Statement:*

Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases

2.*Objectives:*

1.Calculate electricity bills accurately using different slabs and charges.

2.Compare monthly bills and identify unusual increases.

3.*Assumptions:*

1. Electricity consumption is given in units (kWh).

2. Slab rates, fixed charges, surcharges, and rebates are predefined.

4.*Algorithm:*

1.Start

2.Enter monthly electricity consumption.

3. Calculate charges based on slab rates.

4.Add fixed charges and surcharges.

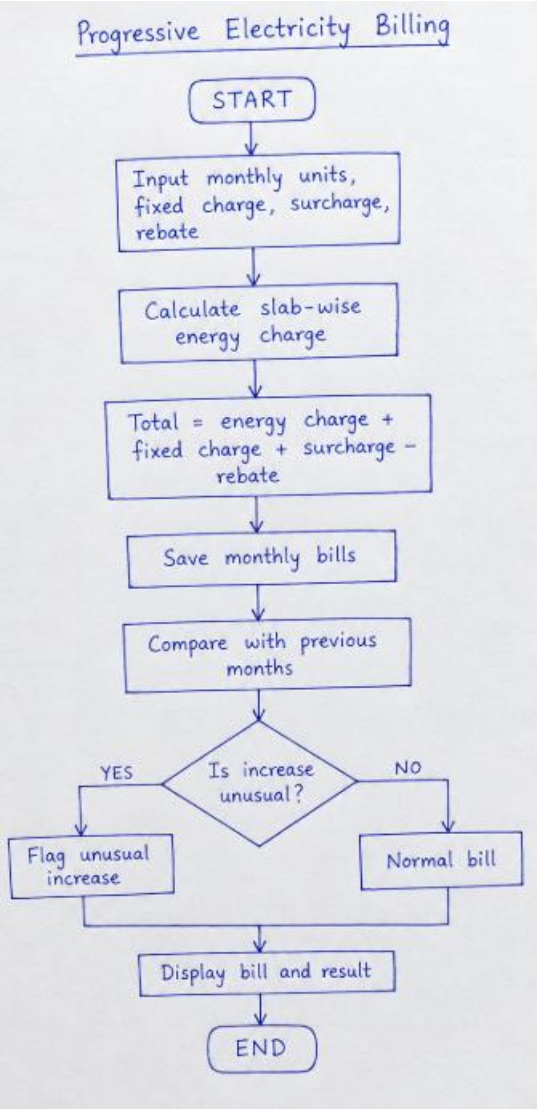
5. Apply eligible rebates.

6. Calculate the final bill.

7. Compare bills across months and identify unusual increases.

8. Stop.

5.*Flow chart:*



6.*Pseudo code:*

```
START
INPUT monthly consumption
CALCULATE slab charges
ADD fixed charges and surcharges
SUBTRACT eligible rebates
DISPLAY final bill
COMPARE monthly bills
IDENTIFY unusual increases
STOP
```

7.*Python Source Code:*

```
units = int(input("Enter electricity units: "))

if units <= 100:

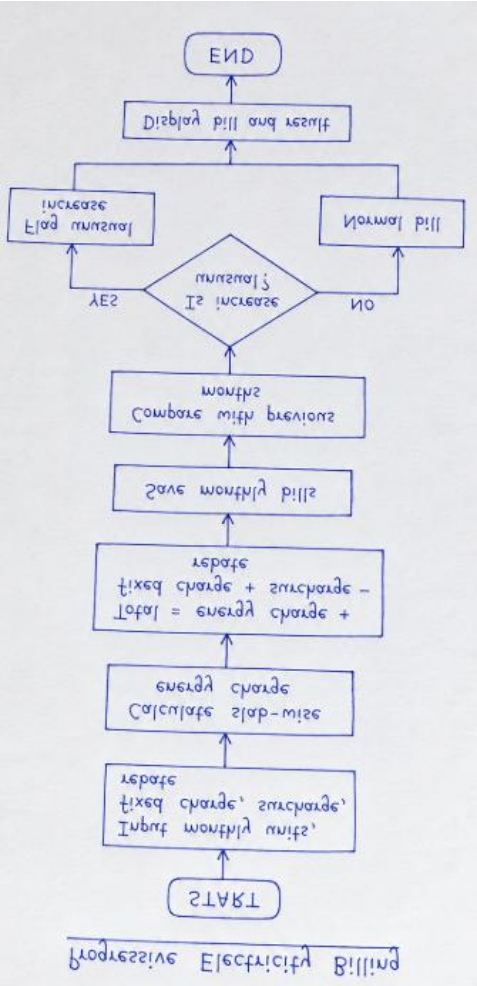
    bill = units * 2
```

```
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
fixed = 50
surcharge = bill * 0.05
rebate = 20 if units < 100 else 0
total = bill + fixed + surcharge - rebate
print("Electricity Bill:", round(total, 2))
```

8.Test Case:

1. For 50 units, the final electricity bill is ₹135.00.
2. For 150 units, the final electricity bill is ₹397.50.
3. For 250 units, the final electricity bill is ₹817.50.

5.*Flow chart:*



6.*Pseudo code:*

```
START
INPUT monthly consumption
CALCULATE slab charges
ADD fixed charges and surcharges
SUBTRACT eligible rebates
DISPLAY final bill
COMPARE monthly bills
IDENTIFY unusual increases
STOP
```

7.*Python Source Code:*

```
units = int(input("Enter electricity units: "))

if units <= 100:

    bill = units * 2

elif units <= 200:

    bill = 100 * 2 + (units - 100) * 3
```

```
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
fixed = 50
surcharge = bill * 0.05
rebate = 20 if units < 100 else 0
total = bill + fixed + surcharge - rebate
print("Electricity Bill:", round(total, 2))
```

8.Test Case:

4. For 50 units, the final electricity bill is ₹135.00.
5. For 150 units, the final electricity bill is ₹397.50.
6. For 250 units, the final electricity bill is ₹817.50.

9.*Code and Output Statement:*

File Edit Format Run Options Window Help

```
units = int(input("Enter electricity units: "))

if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5

fixed = 50
surcharge = bill * 0.05
rebate = 20

total = bill + fixed + surcharge - rebate

print("Slab Charges:", bill)
print("Fixed Charges:", fixed)
print("Surcharge:", surcharge)
print("Rebate:", rebate)
print("Final Electricity Bill:", total)
```

OUTPUT:

```
Enter electricity units: 150
Slab Charges: 350
Fixed Charges: 50
Surcharge: 17.5
Rebate: 20
Final Electricity Bill: 397.5
```

10.*Conclusion:*

The program calculates electricity bills using slab rates, fixed charges, surcharges, and rebates. Monthly comparisons can help identify unusual increases in electricity costs.

CASE 3: Hospital Billing and Patient Classification

1.*Problem Statement:*

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2.*Objectives:*

Calculate the total hospital bill.
Apply eligible concessions and validate billing details.
Classify patients based on their final payable amount

3.*Assumptions:*

1. Consultation, diagnostic, and treatment charges are entered by the user.
2. A 10% concession is provided if the total bill exceeds ₹10,000.
3. All charges must be non-negative
4. Patients are classified as **Low, Medium, or High Payment** based on the final bill.

4.*Algorithm:*

1. Start the program.
2. Read consultation, diagnostic, and treatment charges.
3. Calculate the total bill.
4. Apply a 10% concession if the total exceeds ₹10,000.
5. Calculate the final payable amount.
6. Classify the patient based on the final bill.
7. Display the total bill, concession, final amount, and patient category.
8. Stop the program.

5.*Flowchart:*

CASE 3: Hospital Billing and Patient Classification

1.*Problem Statement:*

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2.*Objectives:*

Calculate the total hospital bill.
Apply eligible concessions and validate billing details.
Classify patients based on their final payable amount

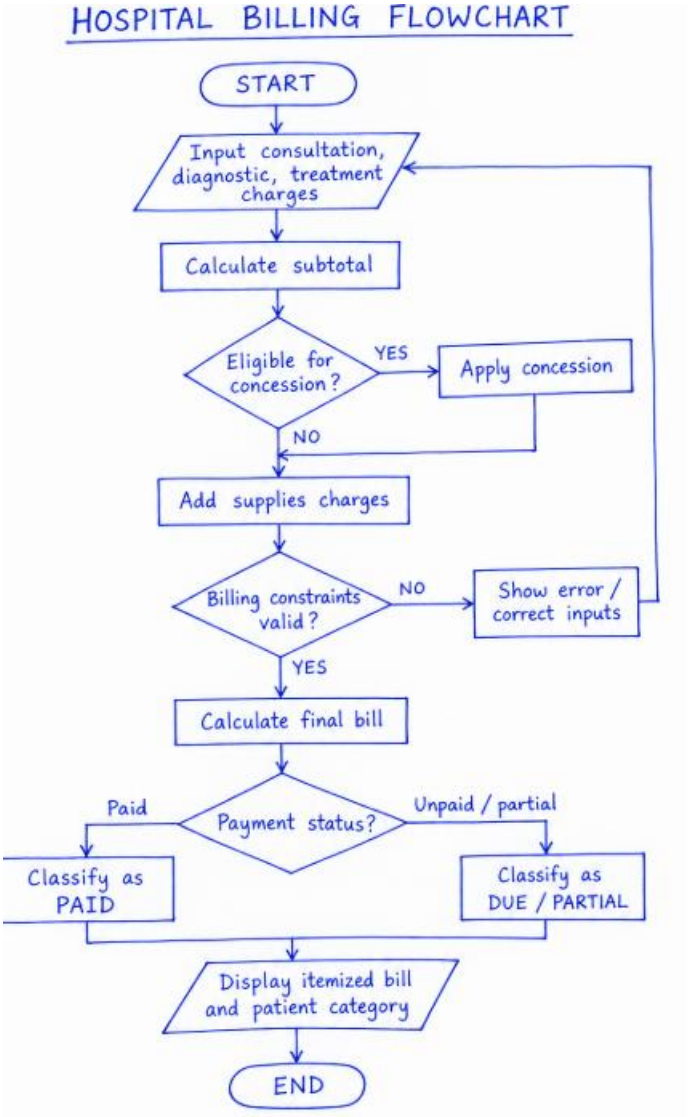
3.*Assumptions:*

5. Consultation, diagnostic, and treatment charges are entered by the user.
6. A 10% concession is provided if the total bill exceeds ₹10,000.
7. All charges must be non-negative
8. Patients are classified as **Low, Medium, or High Payment** based on the final bill.

4.*Algorithm:*

9. Start the program.
10. Read consultation, diagnostic, and treatment charges.
11. Calculate the total bill.
12. Apply a 10% concession if the total exceeds ₹10,000.
13. Calculate the final payable amount.
14. Classify the patient based on the final bill.
15. Display the total bill, concession, final amount, and patient category.
16. Stop the program.

5.*Flowchart:*



6.*Pseudo code:*

```
START
INPUT consultation, diagnostic, treatment
IF any charge < 0
    PRINT "Invalid Charges"
ELSE:
    total = consultation + diagnostic + treatment
    IF total > 10000
        discount = total * 0.10
    ELSE
        discount = 0
    ENDIF
    final_bill = total - discount

    IF final_bill <= 3000
        category = "Low"
    ELSE IF final_bill <= 10000
```

```
        category = "Medium"
    ELSE:
        category = "High"
    ENDIF:

    DISPLAY total, discount, final_bill, category
ENDIF
STOP
```

7.*Python Source Code:*

```
c = float(input("Consultation charges: "))
d = float(input("Diagnostic charges: "))
t = float(input("Treatment charges: "))

if min(c, d, t) < 0:
    print("Invalid Charges")
else:
    total = c + d + t

    if total > 10000:
        discount = total * 0.10
    else:
        discount = 0

    final_bill = total - discount

    if final_bill <= 3000:
        category = "Low"
    elif final_bill <= 10000:
        category = "Medium"
    else:
        category = "High"

    print("Total Bill:", total)
    print("Concession:", discount)
    print("Final Bill:", final_bill)
    print("Patient Category:", category)
```

8.*Test Case:*

The program is tested with different inputs. When the consultation, diagnostic, and treatment charges are ₹500, ₹500, and ₹1,000, the final bill is ₹2,000, and the patient is classified as Low. When the charges are ₹2,000, ₹3,000, and ₹6,000, the total bill is ₹11,000, a 10% concession of ₹1,100 is applied, and the final bill is ₹9,900 (Medium). When the charges are ₹5,000, ₹4,000, and ₹3,000, the final bill after concession is ₹10,800 (High). If any charge is negative, the program displays an invalid charges message.

9.*Code and Output Statement:*

```
c = float(input("Enter consultation charges: "))
d = float(input("Enter diagnostic charges: "))
t = float(input("Enter treatment charges: "))

if c < 0 or d < 0 or t < 0:
    print("Invalid Charges")
else:
    total = c + d + t

    if total > 10000:
        discount = total * 0.10
    else:
        discount = 0

    final_bill = total - discount

    if final_bill <= 3000:
        category = "Low"
    elif final_bill <= 10000:
        category = "Medium"
    else:
        category = "High"

    print("Total Bill:", total)
    print("Concession:", discount)
    print("Final Bill:", final_bill)
    print("Patient Category:", category)
```

OUTPUT:

```
Enter consultation charges: 2000
Enter diagnostic charges: 3000
Enter treatment charges: 6000
Total Bill: 11000.0
Concession: 1100.0
Final Bill: 9900.0
Patient Category: Medium
```

10.*Conclusion:*

The program calculates hospital charges, applies a 10% concession when the total bill exceeds ₹10,000, validates charges, and classifies patients into Low, Medium, or High payment categories.

CASE 4: Employee Payroll and Deduction Analysis

1.*Problem Statement:*

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2.*Objectives:*

Calculate gross salary.
Calculate salary deductions.
Display net salary.

3.*Assumptions:*

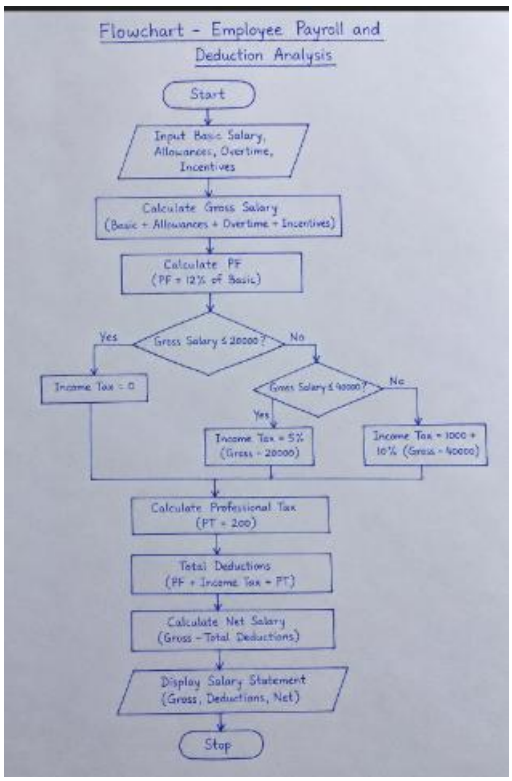
PF = 12% of basic salary.
Professional tax = ₹200.
Overtime rate = 1.5 times hourly pay.
Income tax is calculated using progressive slabs.

4.*Algorithm:*

1.Start.
2.Enter employee salary details.
3.Calculate overtime pay.
4.Calculate gross salary.
5.Calculate PF and income tax.
6.Calculate total deductions.
7.Calculate net salary.
8.Display salary statement.

9.Stop.

5.*Flow chart:*



6.*Pseudo Code:*

```
START
INPUT basic, allowances, overtime, incentives
gross = basic + allowances + overtime + incentives
PF = basic * 0.12
Calculate income tax
deductions = PF + tax + professional tax
net = gross - deductions
DISPLAY gross, deductions, net
STOP
```

7.*Python Source Code:*

```
basic = float(input("Enter basic salary: "))

allowances = float(input("Enter allowances: "))

overtime = float(input("Enter overtime pay: "))

incentives = float(input("Enter incentives: "))

gross = basic + allowances + overtime + incentives

pf = basic * 0.12
```

```
if gross <= 20000:
    tax = 0
elif gross <= 40000:
    tax = (gross - 20000) * 0.05
else:
    tax = 1000 + (gross - 40000) * 0.10
pt = 200
deductions = pf + tax + pt
net = gross - deductions
print("\n--- SALARY STATEMENT ---")
print("Gross Salary:", gross)
print("PF:", pf)
print("Income Tax:", tax)
print("Professional Tax:", pt)
print("Total Deductions:", deductions)
print("Net Salary:", net)
```

8.*Test Case:*

The program is tested with different salary inputs. In the first test case, the basic salary is ₹15,000, allowances are ₹2,000, overtime pay is ₹0, and incentives are ₹500. The gross salary is ₹17,500. In the second test case, the basic salary is ₹25,000, allowances are ₹5,000, overtime pay is ₹1,000, and incentives are ₹2,000. The gross salary is ₹33,000. In the third test case, negative salary input is entered, and the program displays an invalid input message.

9.*Code and Output Statement:*

```
basic = float(input("Enter basic salary: "))
allowances = float(input("Enter allowances: "))
overtime = float(input("Enter overtime pay: "))
incentives = float(input("Enter incentives: "))

gross = basic + allowances + overtime + incentives

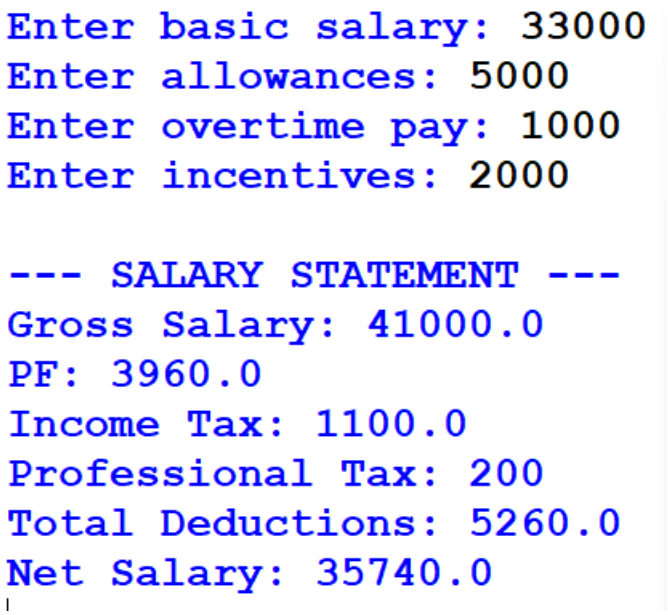
pf = basic * 0.12

if gross <= 20000:
    tax = 0
elif gross <= 40000:
    tax = (gross - 20000) * 0.05
else:
    tax = 1000 + (gross - 40000) * 0.10

pt = 200
deductions = pf + tax + pt
net = gross - deductions

print("\n--- SALARY STATEMENT ---")
print("Gross Salary:", gross)
print("PF:", pf)
print("Income Tax:", tax)
print("Professional Tax:", pt)
print("Total Deductions:", deductions)
print("Net Salary:", net)
```

OUTPUT:



10.*Conclusion:*

The Employee Payroll and Deduction Analysis project calculates the gross salary, total deductions, and net salary of an employee using Python. It uses arithmetic operations and conditional statements to calculate PF, income tax, and professional tax. This program reduces manual calculation errors and generates an accurate salary statement efficiently.

CASE 5: Multi-Plan Telecommunications Cost Optimization

1.*Problem Statement:*

Evaluate several mobile plans against usage, validity, and budget constraints; calculate excess charges and identify the most economical eligible plan.

2.*Objectives:*

To compare different mobile plans.

To calculate additional usage charges.

To check budget and validity.

To find the most economical plan.

3.*Assumptions:*

The customer has a fixed budget of ₹300.

Data usage is 2.5 GB per day.

The minimum validity required is 28 days.

Extra usage charges are calculated based on the selected plan.

4.*Algorithm:*

1.Start.

2.Enter the budget and data usage.

3.Read the available mobile plans.

4.Calculate extra usage charges.

5.Calculate the total cost of each plan.

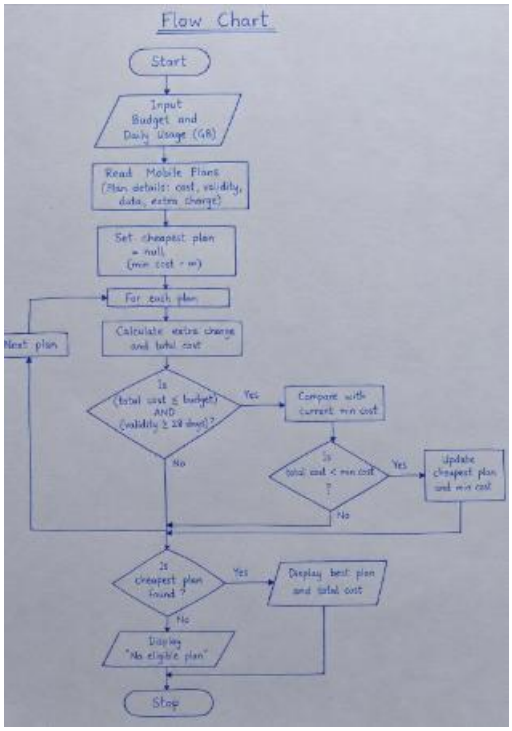
6.Check the budget and validity.

7.Select the cheapest eligible plan.

8.Display the selected plan and total cost.

8.Stop.

5.*Flow chart:*



6.*Pseudo code:*

```
BEGIN
Set budget = 300
Set usage = 2.5 GB per day
```

```
Read mobile plans
FOR each plan
  Calculate extra charges
  Calculate total cost
  IF total cost ≤ budget AND validity ≥ 28 days
    Store eligible plan
  END IF
END FOR
Select the cheapest eligible plan
Display plan name and total cost
END
```

7.*Python source code:*

```
budget = 300

usage = 2.5

plans = [

    {"name": "Plan A", "cost": 199,

     "validity": 28, "data": 2, "extra": 20},

    {"name": "Plan B", "cost": 249,

     "validity": 28, "data": 3, "extra": 20},

    {"name": "Plan C", "cost": 299,

     "validity": 56, "data": 2, "extra": 20}

]

eligible = []

for plan in plans:

    extra_cost = max(0, usage - plan["data"]) * plan["extra"]
* plan["validity"]

    total = plan["cost"] + extra_cost

    if total <= budget and plan["validity"] >= 28:

        eligible.append((plan["name"], total))

if eligible:

    best = min(eligible, key=lambda x: x[1])

    print("Best Plan:", best[0])

    print("Total Cost: Rs.", best[1])
```

```
else:
    print("No eligible plan found")
```

8.*Test case:*

Test Case 1: When the customer's budget is ₹300 and daily data usage is 2.5 GB, Plan A costs ₹339, Plan B costs ₹249, and Plan C costs ₹579. Therefore, Plan B is selected as the cheapest eligible plan.

Test Case 2: When the customer's budget is ₹200 and daily data usage is 2.5 GB, none of the plans satisfies the budget constraint. Therefore, the program displays "No eligible plan found."

Test Case 3: When the customer's budget is ₹300 and daily data usage is 1.5 GB, Plan A costs ₹199, Plan B costs ₹249, and Plan C costs ₹299. Therefore, Plan A is selected as the cheapest eligible plan.

9.*Code and output screen shot:*

```
budget = 300
usage = 2.5
min_validity = 28
plans = [
    {"name": "Plan A", "cost": 199, "validity": 28,
     "data": 2, "extra": 20},
    {"name": "Plan B", "cost": 249, "validity": 28,
     "data": 3, "extra": 20},
    {"name": "Plan C", "cost": 299, "validity": 56,
     "data": 2, "extra": 20}
]
cheapest_plan = None
min_cost = float("inf")
for plan in plans:
    extra_charge = (
        max(0, usage - plan["data"])
        * plan["extra"]
        * plan["validity"]
    )
    total_cost = plan["cost"] + extra_charge
    print("\nPlan:", plan["name"])
    print("Total Cost: Rs.", total_cost)
    if (total_cost <= budget
            and plan["validity"] >= min_validity):
        if total_cost < min_cost:
            min_cost = total_cost
            cheapest_plan = plan["name"]
if cheapest_plan is not None:
    print("\nBest Plan:", cheapest_plan)
    print("Minimum Cost: Rs.", min_cost)
else:
    print("\nNo eligible plan found.")
```

OUTPUT:

```
Plan: Plan A
Total Cost: Rs. 479.0

Plan: Plan B
Total Cost: Rs. 249

Plan: Plan C
Total Cost: Rs. 859.0

Best Plan: Plan B
Minimum Cost: Rs. 249
```

10.*Conclusion:*

The program compares mobile plans, calculates extra charges, checks budget and validity, and identifies the most economical eligible plan.

CASE 6: Railway Fare and Reservation Calculator

1.*Problem statement:*

Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

2.Objectives:

Calculate ticket fares based on distance.

Apply different fares for different travel classes.

Provide age-based concessions.

Add reservation charges.

Calculate cancellation penalties

3.*Assumptions:*

Distance is entered in kilometres.

Travel class is Sleeper or AC.

Children and senior citizens may receive concessions according to the given rules.

Reservation fees and cancellation charges are fixed for this program.

4.*Algorithm:*

1. Start.

2. Enter the number of passengers.

3. Enter each passenger's age, travel distance, and class.

4. Calculate the basic fare.

5.Apply age-based concessions.

6. Add reservation charges.

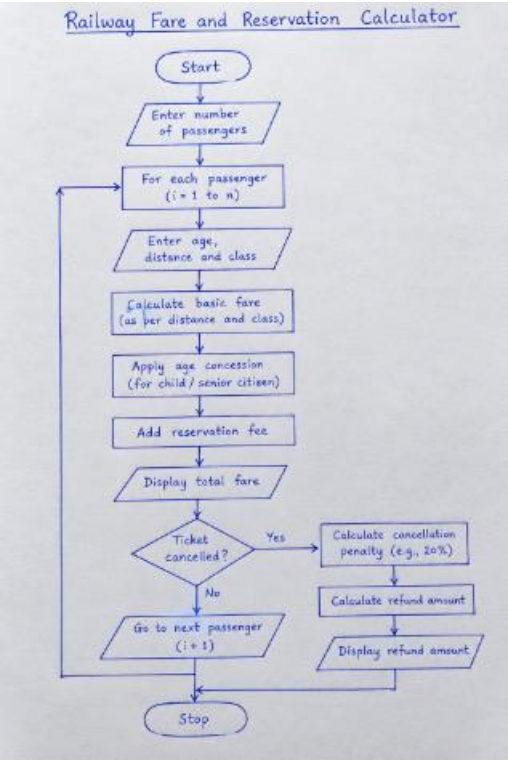
7.Display the final ticket fare.

8.If cancelled, calculate the cancellation penalty.

9.Display the refund amount.

10.Stop.

5.*Flow chart:*



6.*Pseudo code:*

```
START
INPUT number_of_passengers

FOR each passenger
    INPUT age, distance, travel_class
    CALCULATE basic_fare
```

```
   APPLY age_concession
   ADD reservation_fee
   DISPLAY total_fare

   IF ticket_cancelled
      CALCULATE cancellation_penalty
      CALCULATE refund
      DISPLAY refund
    END IF
END FOR
STOP
```

7.Python source code:

```
n = int(input("Enter number of passengers: "))

for i in range(n):

    print("\nPassenger", i + 1)

    age = int(input("Enter age: "))

    distance = float(input("Enter distance in km: "))

    travel_class = input("Enter class (Sleeper/AC): ").lower()

    if travel_class == "sleeper":

        rate = 1

        reservation_fee = 20

    elif travel_class == "ac":

        rate = 2

        reservation_fee = 40

    else:

        print("Invalid class!")

        continue

    fare = distance * rate

    # Age concession

    if age < 5:

        fare = 0

    elif age >= 60:

        fare = fare * 0.90

    total = fare + reservation_fee
```

```
    print("Total Ticket Fare: Rs.", round(total, 2))
    cancel = input("Cancel ticket? (yes/no): ").lower()
    if cancel == "yes":
        penalty = total * 0.20
        refund = total - penalty
        print("Cancellation Penalty: Rs.", round(penalty, 2))
        print("Refund Amount: Rs.", round(refund, 2))
print("\nThank you!")
```

8.*Test case:*

Test Case 1: Input: Age = 30, Distance = 100 km, Class = Sleeper. Expected Output: Total Fare = ₹120.

Test Case 2: Input: Age = 65, Distance = 100 km, Class = Sleeper. Expected Output: Total Fare = ₹110.

9.*Code and Output screenshot:*

```
n = int(input("Enter number of passengers: "))
for i in range(n):
    print("\nPassenger", i + 1)
    age = int(input("Enter age: "))
    distance = float(input("Enter distance in km: "))
    travel_class = input("Enter class (Sleeper/AC): ").lower()
    if travel_class == "sleeper":
        rate = 1
        reservation_fee = 20
    elif travel_class == "ac":
        rate = 2
        reservation_fee = 40
    else:
        print("Invalid class!")
        continue
    fare = distance * rate
    # Age concession
    if age < 5:
        fare = 0
    elif age >= 60:
        fare = fare * 0.90
    total = fare + reservation_fee
    print("Total Ticket Fare: Rs.", round(total, 2))
    cancel = input("Cancel ticket? (yes/no): ").lower()
    if cancel == "yes":
        penalty = total * 0.20
        refund = total - penalty
        print("Cancellation Penalty: Rs.", round(penalty, 2))
        print("Refund Amount: Rs.", round(refund, 2))
print("\nThank you!")
```

OUTPUT:

```
Enter number of passengers: 1

Passenger 1
Enter age: 30
Enter distance in km: 100
Enter class (Sleeper/AC): sleeper
Total Ticket Fare: Rs. 120.0
Cancel ticket? (yes/no): yes
Cancellation Penalty: Rs. 24.0
Refund Amount: Rs. 96.0

Thank you!
```

10.*Conclusion:*

This program calculates railway ticket fares for multiple passengers and demonstrates conditional statements, loops, arithmetic operations, and user input in Python.

CASE 7: Retail Billing with Multiple Discount Rules

1.*Problem Statement:*

Process multiple products, quantities, category specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

2.*Objectives:*

Calculate the price of multiple products.

Calculate the total price using quantity.

Apply taxes based on product categories.

Apply discounts when the bill reaches a specified amount.

Limit the maximum discount.

Generate an itemized invoice.

3.*Assumptions:*

Electronics tax is 18%.

Clothing tax is 12%.

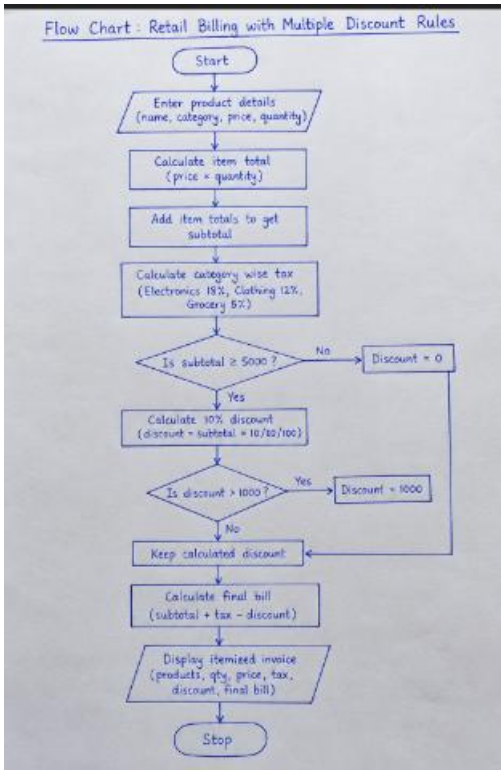
Grocery tax is 5%.

A 10% discount is given when the subtotal is ₹5,000 or more.

The maximum discount is ₹1,000.

All prices are in Indian rupees.

5.*Flow chart:*



6.*Pseudo code:*

```
START


INPUT product details

CALCULATE item totals

CALCULATE subtotal

CALCULATE category taxes


IF subtotal >= 5000

    discount = subtotal * 10 / 100

    IF discount > 1000

        discount = 1000

    ENDIF
```

```
ELSE
    discount = 0
ENDIF

CALCULATE final bill
DISPLAY invoice

STOP
```

7.*Python Source Code:*

```
print("===== RETAIL BILLING SYSTEM =====")
products = [
  ("Mobile", "Electronics", 15000, 1),
  ("Shirt", "Clothing", 1000, 2),
  ("Rice", "Grocery", 500, 2)
]
tax_rates = {
  "Electronics": 18,
  "Clothing": 12,
  "Grocery": 5
}
subtotal = 0
total_tax = 0
print("\n---------- INVOICE ----------")
print("Product  Qty  Price  Total  Tax")

for name, category, price, qty in products:
  item_total = price * qty
  tax = item_total * tax_rates[category] / 100
```

```
    subtotal += item_total
    total_tax += tax
    print(name, qty, price, item_total,
        f"₹{tax:.2f}")
if subtotal >= 5000:
    discount = subtotal * 0.10
    discount = min(discount, 1000)
else:
    discount = 0
final_bill = subtotal + total_tax - discount
print("--------------------------------")
print("Subtotal: ₹", subtotal)
print("Total Tax: ₹", round(total_tax, 2))
print("Discount: ₹", round(discount, 2))
print("Final Bill: ₹", round(final_bill, 2))
print("--------------------------------")
print("Thank you for shopping!")
```

8.*Test case:*

Test Cases: The program is tested with different bill amounts to verify the discount rules. When the subtotal is ₹3,000, no discount is applied. When the subtotal is ₹5,000, a 10% discount of ₹500 is applied. When the subtotal is ₹10,000, a discount of ₹1,000 is applied. When the subtotal is ₹20,000, the calculated discount exceeds ₹1,000, so the maximum discount limit of ₹1,000 is applied. The program also calculates category-wise taxes for electronics, clothing, and groceries and displays the subtotal, total tax, discount, and final bill correctly.

9.*Code and Output Screenshot:*

```
print("===== RETAIL BILLING SYSTEM =====")
products = [
    ("Mobile", "Electronics", 15000, 1),
    ("Shirt", "Clothing", 1000, 2),
    ("Rice", "Grocery", 500, 2)
]
tax_rates = {
    "Electronics": 18,
    "Clothing": 12,
    "Grocery": 5
}
subtotal = 0
total_tax = 0
print("\n---------- INVOICE ----------")
print("Product   Qty   Price   Total   Tax")
for name, category, price, qty in products:
    item_total = price * qty
    tax = item_total * tax_rates[category] / 100
    subtotal += item_total
    total_tax += tax
    print(name, qty, price, item_total,
          f"₹{tax:.2f}")
if subtotal >= 5000:
    discount = subtotal * 0.10
    discount = min(discount, 1000)
else:
    discount = 0
final_bill = subtotal + total_tax - discount
print("-------------------------------")
print("Subtotal: ₹", subtotal)
```

```
print("Subtotal: ₹", subtotal)
print("Total Tax: ₹", round(total_tax, 2))
print("Discount: ₹", round(discount, 2))
print("Final Bill: ₹", round(final_bill, 2))
print("--------------------------------")
print("Thank you for shopping!")
```

OUTPUT:

```
===== RETAIL BILLING SYSTEM =====

---------- INVOICE ----------
Product    Qty    Price    Total    Tax
Mobile 1 15000 15000 ₹2700.00
Shirt 2 1000 2000 ₹240.00
Rice 2 500 1000 ₹50.00
--------------------------------
Subtotal: ₹ 18000
Total Tax: ₹ 2990.0
Discount: ₹ 1000
Final Bill: ₹ 19990.0
--------------------------------
Thank you for shopping!
```

10.*Conclusion:*

This Python program calculates product totals, category-based taxes, discounts, and the final bill. It generates an itemized invoice and ensures that the discount does not exceed the maximum limit.

CASE 8: Manufacturing Cost and Profitability Analysis

1.*Problem Statement:*

Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

2.*Objectives:*

To calculate the material cost.

To calculate labour and machine costs.

To calculate material wastage.

To calculate the total production cost.

To calculate profit and check the target profit.

3.*Assumptions:*

The number of products is greater than zero.

All costs are given in Indian Rupees (₹).

Wastage cost is calculated as a percentage of material cost.

The selling price is given for one product.

The target profit is calculated as a percentage of total production cost.

4.Algorithm:

Start the program.

Enter the number of products.

Enter material, labour, and machine costs per product.

Enter the wastage percentage.

Enter the selling price per product.

Enter the target profit percentage.

Calculate material cost and wastage cost.

Calculate the total production cost.

Calculate total sales and actual profit.

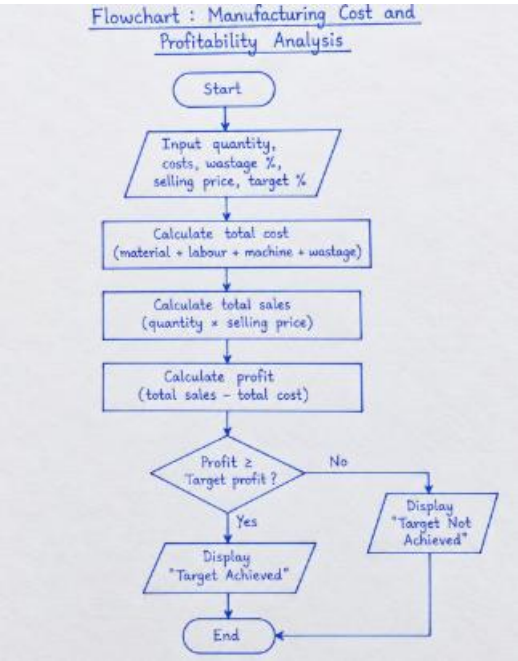
Calculate the target profit.

Check whether the actual profit meets the target.

Display all results.

Stop the program.

5.*Flow chart:*



6.*Pseudo code:*

```
START

INPUT quantity, costs, wastage %, selling price, target %

CALCULATE total cost including wastage

CALCULATE total sales

CALCULATE profit = sales – cost

CALCULATE target profit

IF profit ≥ target profit, DISPLAY "Target Achieved"

ELSE DISPLAY "Target Not Achieved"

DISPLAY cost, sales, and profit

STOP
```

7.*Python Source Code:*

```
q = int(input("Enter quantity: "))

m = float(input("Material cost per unit: "))

l = float(input("Labour cost per unit: "))

mc = float(input("Machine cost per unit: "))

w = float(input("Wastage percentage: "))

sp = float(input("Selling price per unit: "))
```

```
t = float(input("Target profit percentage: "))
material = q * m
wastage = material * w / 100
cost = material + wastage + q * (l + mc)
sales = q * sp
profit = sales - cost
target = cost * t / 100
print("Total Cost:", cost)
print("Total Sales:", sales)
print("Profit:", profit)
if profit >= target:
    print("Target Profit Achieved")
else:
    print("Target Profit Not Achieved")
```

8.*Test case:*

Test Case 1: When the quantity is 100, material cost is ₹50 per unit, labour cost is ₹20 per unit, machine cost is ₹10 per unit, wastage is 5%, and selling price is ₹100 per unit, the total cost is ₹8,250 and the profit is ₹1,750. Since the profit is greater than the target profit of ₹1,237.50 (15%), the result is **"Target Profit Achieved."**

Test Case 2: When the quantity is 100, material cost is ₹50 per unit, labour cost is ₹20 per unit, machine cost is ₹10 per unit, wastage is 5%, and selling price is ₹80 per unit, the total cost is ₹8,250 and the profit is ₹-250. Since the profit is less than the target profit of ₹1,237.50 (15%), the result is **"Target Profit Not Achieved."**

9.*Code and Output statement:*

```
q = int(input("Enter quantity: "))
m = float(input("Material cost per unit: "))
l = float(input("Labour cost per unit: "))
mc = float(input("Machine cost per unit: "))
w = float(input("Wastage percentage: "))
sp = float(input("Selling price per unit: "))
t = float(input("Target profit percentage: "))

material = q * m
wastage = material * w / 100
cost = material + wastage + q * (l + mc)
sales = q * sp
profit = sales - cost
target = cost * t / 100

print("Total Cost:", cost)
print("Total Sales:", sales)
print("Profit:", profit)

if profit >= target:
    print("Target Profit Achieved")
else:
    print("Target Profit Not Achieved")
```

OUTPUT:

```
Enter quantity: 100
Material cost per unit: 50
Labour cost per unit: 20
Machine cost per unit: 10
Wastage percentage: 5
Selling price per unit: 100
Target profit percentage: 15
Total Cost: 8250.0
Total Sales: 10000.0
Profit: 1750.0
Target Profit Achieved
```

10.*Conclusion:*

This program calculates the manufacturing cost, material wastage, labour charges, machine costs, total sales, and profit. It also checks whether the production meets the specified profit target. Python makes these calculations simple and accurate.

CASE 9: Multi-Currency Travel Budget Analysis

1.*Problem statement:*

Convert travel expenses using supplied exchange rates, incorporate conversion fees, compare planned spending against a budget, and identify the largest expense category.

2.*Objectives:*

Convert foreign currencies into Indian Rupees.

Calculate conversion fees.

Compare expenses with the budget.

Find the highest expense category.

3.*Assumptions:*

The budget is ₹50,000.

Exchange rates are given by the user.

Conversion fees are 2%.

All expenses are valid positive amounts.

4.*Algorithm:*

1.Start.

2.Enter travel expenses.

3.Enter exchange rates.

4.Convert expenses into INR.

5.Calculate conversion fees.

6.Calculate total expenses.

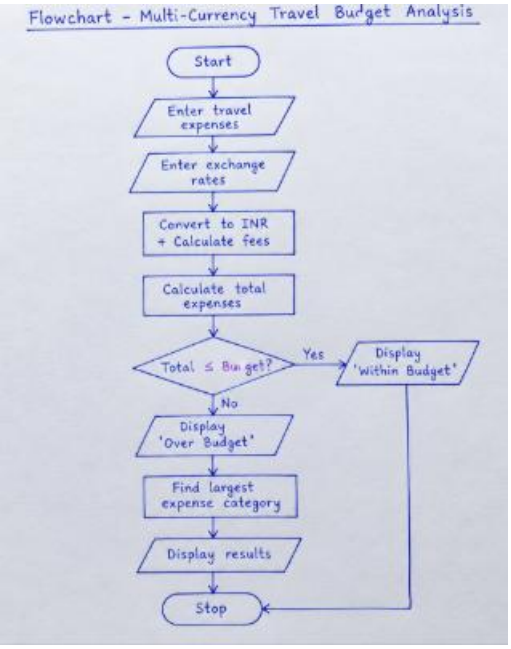
7.Compare expenses with the budget.

8. Find the largest expense category.

9.Display the results.

10.Stop.

5.*Flow chart:*



6.*Pseudo code:*

```
START
INPUT travel expenses
INPUT exchange rates
FOR each expense
    Convert expense into INR
    Calculate conversion fee
    Calculate final expense
END FOR
Calculate total expenses
```

```
Compare total with budget
Find largest expense category
DISPLAY results
STOP
```

7.*Python Source Code:*

```
budget = 50000

expenses = {"Hotel": 200, "Food": 100, "Shopping": 50}

rates = {"Hotel": 85, "Food": 95, "Shopping": 110}

total = 0

for category in expenses:

    amount = expenses[category] * rates[category]

    fee = amount * 0.02

    total += amount + fee

    print(category, "=", round(amount + fee, 2))

print("Total Expense =", round(total, 2))

if total <= budget:

    print("Within Budget")

else:

    print("Over Budget")

print("Largest Expense =", max(

    expenses, key=lambda x: expenses[x] * rates[x]

))
```

8.*Test case:*

Test Cases: When the total travel expense is ₹32,640 and the budget is ₹50,000, the program displays “Within Budget.” If the total expense exceeds ₹50,000, it displays “Over Budget.” The program also converts expenses into INR, adds a 2% conversion fee, and identifies the largest expense category.

9.*Code and output screenshot:*

```
budget = 50000
expenses = {"Hotel": 200, "Food": 100, "Shopping": 50}
rates = {"Hotel": 85, "Food": 95, "Shopping": 110}
total = 0
for category in expenses:
    amount = expenses[category] * rates[category]
    fee = amount * 0.02
    total += amount + fee
    print(category, "=", round(amount + fee, 2))
print("Total Expense =", round(total, 2))
if total <= budget:
    print("Within Budget")
else:
    print("Over Budget")
print("Largest Expense =", max(
    expenses, key=lambda x: expenses[x] * rates[x]
))
```

OUTPUT:

```
Hotel = 17340.0
Food = 9690.0
Shopping = 5610.0
Total Expense = 32640.0
Within Budget
Largest Expense = Hotel
```

10.*Conclusion:*

This program converts travel expenses into Indian Rupees, calculates conversion fees, checks the budget, and identifies the largest expense category.

CASE 10: Campus Event Financial Planning

1.*Problem Statement:*

Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

2.*Objectives:*

Calculate total event costs.

Find the contribution per participant.

Calculate the sponsorship shortfall.

Find the minimum registrations required to break even.

3.*Assumptions:*

- The event is conducted for college students.
- All costs are given in rupees.
- Each participant pays a fixed registration fee.
- Sponsorship is a fixed amount.

4.*Algorithm:*

1. Start.

2. Enter venue, equipment, catering, transport, and certificate costs.

3.Calculate the total cost.

4.Enter sponsorship and registration fee.

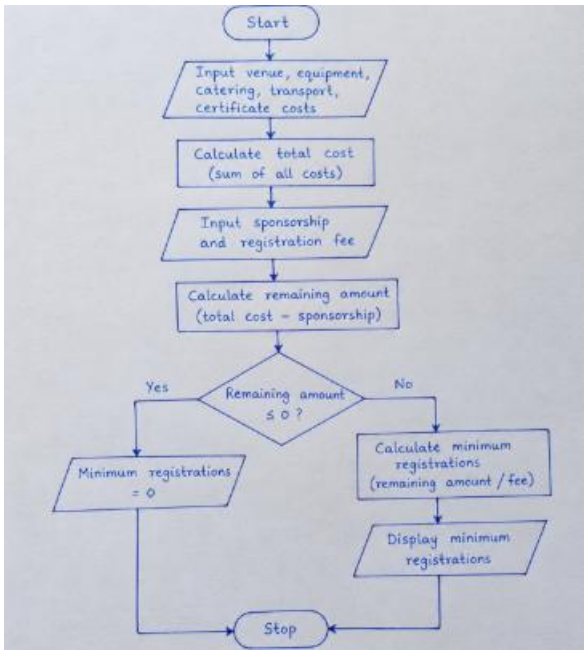
5.Calculate the remaining amount.

6.Calculate the minimum registrations required.

7.Display the results.

8.Stop.

5.*Flow chart:*



6.*Pseudo Code:*

```
START
INPUT all event costs
total = sum of all costs
INPUT sponsorship, registration_fee
remaining = total - sponsorship
IF remaining <= 0
   PRINT 0 registrations
ELSE
   registrations = CEILING(remaining / registration_fee)
   PRINT registrations
END IF
STOP
```

7.*Python Source Code:*

```
import math

venue = 10000
equipment = 5000
catering = 20000
transport = 5000
certificates = 10000

total = venue + equipment + catering + transport +
certificates
sponsorship = 20000
fee = 500

remaining = total - sponsorship

print("Total Cost:", total)
print("Sponsorship:", sponsorship)
print("Remaining Amount:", remaining)

if remaining <= 0:
    print("Minimum Registrations: 0")
else:
    registrations = math.ceil(remaining / fee)
    print("Minimum Registrations:", registrations)
```

8.*Test case:*

Test Cases: In Test Case 1, the total cost is ₹50,000, sponsorship is ₹20,000, and the registration fee is ₹500. The remaining amount is ₹30,000, so 60 registrations are required. In Test Case 2, if sponsorship is ₹50,000, it covers the total cost, so 0 registrations are required to cover expenses. In Test Case 3, if the total cost is ₹40,000, sponsorship is ₹10,000, and the registration fee is ₹300, then 100 registrations are required to cover the remaining amount.

9.*Code and output screenshot:*

```
import math
venue = 10000
equipment = 5000
catering = 20000
transport = 5000
certificates = 10000
total = venue + equipment + catering + transport + certificates
sponsorship = 20000
fee = 500
remaining = total - sponsorship
print("Total Cost:", total)
print("Sponsorship:", sponsorship)
print("Remaining Amount:", remaining)
if remaining <= 0:
    print("Minimum Registrations: 0")
else:
    registrations = math.ceil(remaining / fee)
    print("Minimum Registrations:", registrations)
```

OUTPUT:

```
Total Cost: 50000
Sponsorship: 20000
Remaining Amount: 30000
Minimum Registrations: 60
```

10.*Conclusion:*

The program calculates total event expenses, sponsorship balance, and the minimum number of registrations needed to cover the remaining cost.